

SRE.ai

Production Build Guide

Day-by-Day Engineering Execution Plan

Solo Builder • Full-Time • AWS Free Tier • September 2026

This document covers only one thing: how to build SRE.ai in production, day by day, decision by decision. No market analysis. No business strategy. Pure engineering execution.

Build Overview

Attribute	Detail
Total build time	10 weeks (full-time solo)
Working hours	8 hours/day, 6 days/week
Target: MVP done by	Week 8
Target: First real user by	Week 10
Deployment	AWS Free Tier → paid on first customer
Primary stack	NestJS, TypeScript, PostgreSQL, pgvector, Redis, SQS, BullMQ
LLM	OpenAI GPT-4o (diagnosis) + text-embedding-3-small (vectors)
Frontend	Next.js 14 App Router + Tailwind CSS
Local dev	Docker Compose (all services local before AWS touch)

The 6 Phases

Phase	Name	Duration
0	Foundation — monorepo, DB, auth, Docker	Week 1
1	Ingestion — webhooks, SQS, deduplication	Week 2
2	Diagnosis Engine — context, vectors, LLM agent	Week 3-4 (hardest)
3	Action Layer — confidence gating, auto-execute, rollback	Week 5-6
4	Memory & Learning — incident store, post-mortems, runbooks	Week 7
5	Dashboard & Polish — Next.js UI, analytics, onboarding	Week 8-10

Week 3-4 (Diagnosis Engine) is the hardest and most important phase. Budget more time here. Do not rush it. The LLM reasoning quality determines whether this product is trustworthy or just another noisy AI tool.

Environment Setup — Before Day 1

Do this before you write a single line of application code. A solid local environment saves hours of debugging later.

Tools to Install

- Node.js 20 LTS (use nvm)
- pnpm (faster than npm for monorepos)
- Docker Desktop
- PostgreSQL client: TablePlus or DBeaver
- AWS CLI v2 — configured with free tier account
- Git — with SSH key configured
- VSCode with extensions: ESLint, Prettier, REST Client, Docker

AWS Free Tier Services You'll Use

Service	Free Tier Limit
EC2 t2.micro / t3.micro	750 hours/month (12 months)
RDS PostgreSQL db.t3.micro	750 hours/month + 20GB storage
SQS	1 million requests/month free forever
ElastiCache (Redis) cache.t3.micro	750 hours/month (12 months)
S3	5GB storage, 20k GET, 2k PUT/month
CloudWatch	10 custom metrics, 5GB logs/month
SES (email)	62,000 emails/month if sending from EC2

Keep everything local with Docker Compose for the first 4 weeks. Only move to AWS when you're ready to test end-to-end with real webhooks. This saves money and speeds up development loops.

Local Docker Compose Stack

```
# docker-compose.yml
version: '3.8'
services:
  postgres:
    image: pgvector/pgvector:pg16
    ports: ['5432:5432']
    environment:
      POSTGRES_DB: sreai
      POSTGRES_USER: sreai
      POSTGRES_PASSWORD: sreai_local
    volumes: ['postgres_data:/var/lib/postgresql/data']
```

```

redis:
  image: redis:7-alpine
  ports: ['6379:6379']

localstack:
  image: localstack/localstack
  ports: ['4566:4566']
  environment:
    SERVICES: sqs,sns

volumes:
  postgres_data:

```

PHASE 0: Foundation — Monorepo, DB, Auth, Docker Week 1 — Days 1-6

Goal: a working NestJS monorepo with auth, multi-tenancy, database migrations, and local Docker running. No business logic yet — just the skeleton that everything else plugs into.

Day 1 Monorepo Setup + NestJS Scaffold [FOUNDATION]

Morning (4 hours) — Monorepo scaffold

```

# Bootstrap
pnpm init
pnpm add -g @nestjs/cli
nest new api-gateway --strict
nest new ingestion-service --strict
nest new diagnosis-service --strict
nest new action-service --strict

```

Afternoon (4 hours) — Shared packages + config

- Create packages/shared — DTOs, constants, types used across all services
- Create packages/database — TypeORM entities, migrations, base repository
- Setup pnpm workspaces in root package.json
- Configure ESLint + Prettier across all packages
- Setup tsconfig.base.json with path aliases

- ☐ Monorepo structure created
- ☐ All 4 services scaffold and run
- ☐ Shared packages configured
- ☐ ESLint + Prettier passing

Day 2 Database Setup + Migrations [FOUNDATION]

Morning — PostgreSQL + pgvector + TypeORM

```

pnpm add @nestjs/typeorm typeorm pg
# Enable pgvector
CREATE EXTENSION IF NOT EXISTS vector;

```

- Configure TypeORM with migrations enabled (not synchronize: true in prod)
- Create base entity with id (UUID), createdAt, updatedAt

- Setup migration CLI script in package.json

Afternoon — Core schemas (migration files)

```
-- Migration: 001_create_tenants
CREATE TABLE tenants (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        VARCHAR(100) NOT NULL,
  slug        VARCHAR(50) UNIQUE NOT NULL,
  plan        VARCHAR(20) DEFAULT 'hobby',
  created_at  TIMESTAMPTZ DEFAULT NOW()
);
```

```
-- Migration: 002_create_incidents
CREATE TABLE incidents (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID REFERENCES tenants(id) ON DELETE CASCADE,
  alert_source VARCHAR(50) NOT NULL,
  service_name VARCHAR(100) NOT NULL,
  severity    VARCHAR(5) NOT NULL,
  title       TEXT NOT NULL,
  raw_alert   JSONB NOT NULL,
  status      VARCHAR(20) DEFAULT 'detecting',
  opened_at   TIMESTAMPTZ DEFAULT NOW(),
  resolved_at TIMESTAMPTZ,
  mttr_seconds INTEGER,
  embedding   vector(1536)
);
```

```
-- Migration: 003_create_diagnoses
CREATE TABLE diagnoses (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  incident_id UUID REFERENCES incidents(id) ON DELETE CASCADE,
  hypothesis  TEXT NOT NULL,
  citations   JSONB NOT NULL DEFAULT '[]',
  confidence  DECIMAL(4,3) NOT NULL,
  context_used JSONB,
  similar_ids UUID[],
  created_at  TIMESTAMPTZ DEFAULT NOW()
);
```

```
-- Migration: 004_create_actions
CREATE TABLE actions (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  incident_id UUID REFERENCES incidents(id) ON DELETE CASCADE,
  action_type VARCHAR(30) NOT NULL,
  action_taken TEXT NOT NULL,
  payload     JSONB,
  outcome     VARCHAR(20),
  executed_at TIMESTAMPTZ DEFAULT NOW(),
  rolled_back_at TIMESTAMPTZ
);
```

```
-- Migration: 005_create_integrations
CREATE TABLE integrations (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID REFERENCES tenants(id) ON DELETE CASCADE,
  type        VARCHAR(50) NOT NULL,
  config      JSONB NOT NULL DEFAULT '{}',
  active      BOOLEAN DEFAULT true,
  created_at  TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(tenant_id, type)
);
```

- ☐ Docker Compose running with postgres + redis
- ☐ pgvector extension enabled
- ☐ All 5 migrations run successfully
- ☐ TypeORM entities created for each table

Day 3 Auth + Multi-Tenancy [FOUNDATION]

Morning — JWT Auth

```
pnpm add @nestjs/jwt @nestjs/passport passport passport-jwt bcrypt
pnpm add -D @types/passport-jwt @types/bcrypt
```

- AuthModule with register (email + password) and login endpoints
- JWT access token (15 min expiry) + refresh token (7 days, stored in httpOnly cookie)
- JwtAuthGuard as global guard — all routes protected by default
- @Public() decorator to opt-out specific routes (webhook endpoints need this)

Afternoon — Multi-tenancy middleware

- TenantMiddleware — extract tenant from JWT, attach to request context
- TenantId injected into all repository calls via TypeORM query scope
- Row-level isolation — every query automatically filters by tenant_id
- API key auth for webhook endpoints (separate from JWT — no login required)

```
// Every repository call automatically scoped
async findAll(): Promise<Incident[]> {
  return this.repo.find({
    where: { tenantId: this.tenantContext.id }
  });
}
```

- ☐ Register + login endpoints working
- ☐ JWT guard protecting all routes
- ☐ Multi-tenant isolation verified (tenant A cannot see tenant B data)
- ☐ API key auth working for webhook endpoints

Day 4 SQS Setup (LocalStack) + BullMQ [FOUNDATION]

Morning — LocalStack SQS

```
pnpm add @aws-sdk/client-sqs
```

- Create 3 SQS queues in LocalStack via aws CLI or SDK on startup:
 - sreai-incidents-p1 (high priority — P1 severity alerts)
 - sreai-incidents-p2 (normal — P2/P3 alerts)
 - sreai-incidents-dlq (dead letter — after 3 failed attempts)
- SqsModule — wrapper service for enqueue, dequeue, delete, visibility
- DLQ configured: maxReceiveCount = 3, then move to DLQ

Afternoon — BullMQ for diagnosis jobs

```
pnpm add bullmq @nestjs/bullmq
```

- DiagnosisQueue — BullMQ queue backed by Redis
- DiagnosisWorker — processes diagnosis jobs with 3-attempt exponential backoff
- Job priority: P1 incidents get priority 1 (highest), P2 get priority 5
- Job timeout: 120 seconds max per diagnosis attempt
- Failed jobs after 3 attempts → alert owner via Slack with raw alert context

- ☐ 3 SQS queues running in LocalStack
- ☐ BullMQ connected to Redis
- ☐ Test job enqueued and processed end-to-end
- ☐ DLQ receiving failed messages after 3 retries

Day 5 Logging + Error Handling + Config [FOUNDATION]

Morning — Structured logging

```
pnpm add winston nest-winston
```

- WinstonLogger replacing default NestJS logger across all services
- Every log line includes: timestamp, service name, traceId, tenantId, level, message
- TraceId generated per request, propagated through all service calls via headers
- Log levels: error (always), warn (always), info (prod), debug (dev only)
- Local: console transport. AWS: CloudWatch Logs transport

Afternoon — Global error handling + validation

```
pnpm add class-validator class-transformer
```

- GlobalExceptionHandler — catches all unhandled exceptions, logs with traceId, returns structured error response
- ValidationPipe globally applied — auto-validates all DTOs with class-validator
- All endpoints return consistent response shape: { success, data, error, traceId }
- Sentry integration for error tracking (free tier: 5k errors/month)

- ☐ All logs include traceId and tenantId
- ☐ Global exception filter catching and logging all errors
- ☐ DTO validation working on all endpoints
- ☐ Sentry receiving test errors

Day 6 Docker + CI/CD skeleton [FOUNDATION]

Full day — Dockerfile + GitHub Actions

```
# Dockerfile (multi-stage)
FROM node:20-alpine AS builder
WORKDIR /app
COPY package.json pnpm-lock.yaml ./
RUN npm install -g pnpm && pnpm install --frozen-lockfile
COPY . .
RUN pnpm build

FROM node:20-alpine AS runner
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
CMD ['node', 'dist/main.js']
```

- Dockerfile for each service (multi-stage, minimal image)
- docker-compose.prod.yml — production compose with env vars from .env.prod
- GitHub Actions pipeline: lint → test → build → (deploy placeholder)
- Secrets in GitHub: DATABASE_URL, REDIS_URL, OPENAI_API_KEY, etc.
- .env.example committed — .env never committed

- ☐ All services build and run in Docker
- ☐ GitHub Actions pipeline runs on push to main
- ☐ No secrets in codebase

Phase 0 complete. You now have a production-grade skeleton. Every engineer on a team of 10 would be comfortable working in this codebase. That foundation pays dividends every day from here.

PHASE 1: Ingestion — Webhooks, SQS, Deduplication Week 2 — Days 7-12

Goal: SRE.ai can receive real alerts from Prometheus, Sentry, and Grafana, deduplicate them, enrich them, and place them in the correct priority SQS queue for diagnosis.

Day 7 Webhook Receivers — Prometheus + Generic [INGESTION]

Morning — Prometheus Alertmanager receiver

```
POST /webhooks/prometheus/:tenantApiKey
```

Prometheus Alertmanager sends a JSON payload when an alert fires. Parse it, normalise it into SRE.ai's internal AlertDTO, validate severity, and enqueue.

```
// AlertDTO — internal normalised format
class AlertDTO {
  source: 'prometheus' | 'sentry' | 'grafana' | 'cloudwatch' | 'generic'
  tenantId: string
  serviceId: string
  severity: 'P1' | 'P2' | 'P3'
  title: string
  description: string
  labels: Record<string, string>
  firedAt: Date
  resolvedAt?: Date
  rawPayload: object
}
```

Severity mapping — Prometheus

- labels.severity = 'critical' → P1
- labels.severity = 'warning' → P2
- everything else → P3

Afternoon — Generic webhook + Sentry receiver

- Generic webhook: accepts any JSON, user maps fields to AlertDTO via integration config
- Sentry: parse issue.created and issue.resolved events, map error rate to severity
- All receivers return 200 immediately — processing is async
- Return 401 for invalid API key, 400 for malformed payload

- ☐ Prometheus webhook receiving and parsing real payloads
- ☐ Sentry webhook receiving issue events
- ☐ Generic webhook with field mapping working
- ☐ All receivers return 200 < 100ms

Day 8 Deduplication + Enrichment [INGESTION]

Morning — Deduplication

The same alert can fire from multiple sources simultaneously. Without deduplication, SRE.ai would create 3 diagnosis jobs for 1 real incident.

```
// Deduplication key: hash of (tenantId + serviceId + alertTitle + 5min window)
const dedupeKey = `dedupe:${tenantId}:${serviceId}:${hash(title)}`;
const exists = await redis.get(dedupeKey);
if (exists) return; // skip - already processing
await redis.setex(dedupeKey, 300, '1'); // 5 min window
```

- Redis-backed deduplication with 5-minute sliding window
- Deduplication key = SHA256(tenantId + serviceId + normalised title)
- Duplicate alerts still logged for audit — just not re-processed
- Alert storm protection: if > 20 alerts from same service in 1 minute → group into single P1 incident

Afternoon — Alert enrichment

- Pull service metadata from tenant's integration config (owner, repo, runbook URL)
- Check if a deploy happened in last 30 min via GitHub integration (if configured)
- Attach enrichment to AlertDTO before enqueueing
- If no enrichment available — enqueue anyway with partial data

- ☐ Duplicate alerts within 5 min window rejected
- ☐ Alert storm grouping working (20+ alerts → 1 incident)
- ☐ Service metadata attached to alerts
- ☐ GitHub deploy correlation working

Day 9 SQS Enqueue + Incident Creation [INGESTION]

Morning — SQS priority routing

- P1 alerts → sreai-incidents-p1 queue (maxReceiveCount: 3)
- P2/P3 alerts → sreai-incidents-p2 queue (maxReceiveCount: 3)
- Failed after 3 → sreai-incidents-dlq
- Message body: serialised AlertDTO + enrichment + traceId
- Message attributes: tenantId, severity (for filtering)

Afternoon — Incident record creation

- When alert enqueued: create incident record in DB with status = 'detecting'
- Incident ID returned in enqueue response (for correlation)
- SQS message carries incident ID — diagnosis service updates same record
- Emit real-time update via Redis pub/sub → dashboard WebSocket (placeholder for now)

- ☐ P1 and P2/P3 routing to correct queues

- ☐ Incident created in DB on every unique alert
- ☐ Incident status = 'detecting' on creation
- ☐ DLQ receiving messages after 3 failures

Day 10 Grafana + CloudWatch receivers [INGESTION]

Full day — Two more sources

- Grafana: parse Grafana webhook payload (different schema from Prometheus)
 - — evalMatches array → extract metric name and value
 - — state: 'alerting' = firing, state: 'ok' = resolved
 - CloudWatch: SQS → SNS → SRE.ai bridge
 - — CloudWatch alarm triggers SNS topic
 - — SNS HTTP subscription POSTs to SRE.ai /webhooks/cloudwatch endpoint
 - — Parse AlarmName, NewStateValue, NewStateReason
 - Resolve detection: when Prometheus/Grafana sends a resolved alert → update incident status to 'resolved', calculate mttr_seconds
- ☐ Grafana alerts parsing correctly
 - ☐ CloudWatch alarms reaching SRE.ai via SNS
 - ☐ Alert resolution updating incident status and MTTR

Day 11 Integration Management API [INGESTION]

Full day — CRUD for integrations

```
POST /integrations      - connect new integration
GET  /integrations      - list active integrations
PATCH /integrations/:id - update config
DELETE /integrations/:id - disconnect
POST  /integrations/:id/test - send test alert
```

- Integration config stored encrypted in DB (AES-256 for API keys, tokens)
 - Test endpoint: sends a synthetic test alert through the full ingestion pipeline
 - Webhook URL shown to user after creating integration: /webhooks/prometheus/THEIR_API_KEY
 - API key rotation: generate new key, old key valid for 24 hours
- ☐ All integration CRUD endpoints working
 - ☐ Config encrypted at rest
 - ☐ Test alert flows through full ingestion pipeline
 - ☐ Webhook URLs correctly generated per tenant

Day 12 Ingestion E2E test + load test [INGESTION]

Full day — Testing and hardening

- Write e2e tests: send real Prometheus payloads, verify incident created in DB
- Duplicate alert test: send same alert twice in 4 min — verify one incident created
- Alert storm test: send 25 alerts in 60s — verify grouped into 1 P1 incident
- Load test with autocannon: 100 webhook requests/second for 30 seconds
- Target: p95 < 50ms response time at 100 RPS
- Fix any race conditions found in deduplication under load

- ☐ E2E tests passing for all 4 sources
- ☐ Load test: p95 < 50ms at 100 RPS
- ☐ No duplicate incidents in DB after deduplication tests

Phase 1 complete. Real alerts now flow into SRE.ai reliably. Next: make the system intelligent.

PHASE 2: Diagnosis Engine — The Hard Part Week 3-4 — Days 13-24

Goal: when an alert fires, SRE.ai automatically collects all relevant context (logs, metrics, deploys, similar incidents) and produces a grounded root-cause hypothesis with citations and a confidence score. This is the product's core value — get this wrong and nothing else matters.

This is the most complex phase. The LLM reasoning quality and citation enforcement are what separate SRE.ai from a toy project. Do not ship a diagnosis that can hallucinate. Take the full 2 weeks.

Day 13 Context Collector — Architecture [DIAGNOSIS]

The context collector is a fan-out service. When a diagnosis job starts, it simultaneously fires 5-6 async calls to collect context, waits for all to complete (or timeout), then passes the aggregated context to the LLM agent.

```
interface DiagnosisContext {
  incident: Incident
  logs: LogEntry[]           // last 200 lines from service
  metrics: MetricSnapshot    // CPU, mem, latency, error rate at alert time
  recentDeploy?: DeployInfo // last deploy if within 30 min
  dependencies: ServiceHealth[] // upstream/downstream status
  similarIncidents: PastIncident[] // top 3 semantic matches
}
```

Collector services to build:

- LogCollector — pulls last N log lines from CloudWatch Logs or Loki
- MetricsCollector — pulls metrics snapshot from Prometheus or Grafana API
- DeployCollector — queries GitHub API for recent commits to service repo
- DependencyCollector — pings health check endpoints of related services
- SimilarIncidentCollector — vector search over past incidents (Day 17)

Fan-out pattern with timeout:

```
const [logs, metrics, deploy, deps] = await Promise.allSettled([
  this.logCollector.collect(incident, { lines: 200, timeWindow: '15m' }),
  this.metricsCollector.collect(incident, { timeWindow: '30m' }),
  this.deployCollector.collect(incident),
  this.dependencyCollector.collect(incident),
]);
// allSettled — never fails even if one collector is down
// partial context is better than no diagnosis
```

- ☐ DiagnosisContext interface defined
- ☐ Fan-out pattern with Promise.allSettled implemented
- ☐ Each collector has 10s individual timeout
- ☐ Partial context handled gracefully

Morning — Log Collector

```
pnpm add @aws-sdk/client-cloudwatch-logs
```

- CloudWatch Logs: FilterLogEvents API — filter by service name, time range
- Pull last 200 log lines from 15 minutes before alert fired
- Parse log lines: extract timestamp, level, message, any error stack traces
- Truncation: if > 200 lines, keep first 50 (startup) + last 150 (recent activity)
- Format for LLM: clean, numbered, with timestamps stripped to save tokens

Afternoon — Metrics Collector

- Prometheus API: query_range for CPU, memory, request rate, error rate, latency
- Time range: 30 minutes before alert → alert time
- Format as summary: 'CPU peaked at 94% at 03:12 UTC, baseline was 30%'
- Grafana fallback: use Grafana HTTP API if Prometheus not configured
- If no metrics integration: skip gracefully, note absence in context

- ☐ Log collector pulling real CloudWatch logs
- ☐ Metrics collector returning formatted summaries
- ☐ Both handle timeouts and unavailable sources gracefully

Morning — Deploy Collector (GitHub)

```
pnpm add @octokit/rest
```

- GitHub API: list commits to service repo in last 30 minutes
- If commit found: extract author, message, files changed, PR link
- High-signal deploy markers: changes to config files, environment files, dependency files
- Format: 'Deploy detected 8 minutes before alert: PR #142 by @alice — updated Redis connection pool config'
- No GitHub integration: skip gracefully

Afternoon — Dependency Collector

- Ping health check endpoint (GET /health or /ping) of each related service
- Related services defined in tenant's service config — stored in integrations table
- Timeout: 3 seconds per health check
- Format: 'auth-service: healthy (120ms) | payments-service: unhealthy (timeout) | db: healthy (8ms)'
- Unhealthy dependency is high-signal — often the root cause

- ☐ Deploy collector finding commits within 30 min window
- ☐ Dependency health checks running in parallel
- ☐ All collectors integrated into DiagnosisContext

The system prompt is the most important piece of engineering in this entire product. A bad system prompt produces hallucinated, untrustworthy diagnoses. A good one produces grounded, citable, actionable diagnoses that engineers trust.

System prompt design — principles:

- Role: 'You are an expert Site Reliability Engineer'
- Task: diagnose the root cause of a production incident
- Constraint: every factual claim MUST be followed by [SOURCE: <type>, <reference>]
- Constraint: if you cannot find evidence for a claim — do not make the claim
- Constraint: output ONLY valid JSON matching the DiagnosisOutput schema
- Constraint: confidence score must reflect evidence quality, not your best guess

```
// DiagnosisOutput schema
{
  hypothesis: string,           // 1-3 sentence root cause
  confidence: number,          // 0.0 to 1.0
  evidence: [{
    claim: string,             // specific factual claim
    source: string,            // 'logs' | 'metrics' | 'deploy' | 'dependency' |
    'similar_incident'
    reference: string,         // specific log line, metric value, commit SHA, etc.
  }],
  recommended_action: string,  // specific, executable action
  action_tier: 'auto' | 'draft' | 'escalate',
  reasoning: string,           // how you reached this conclusion
}
```

Context assembly for prompt:

- Token budget: 8000 tokens for context (GPT-4o 128k context window)
- Priority order if truncating: similar incidents > logs > metrics > deploy info > dependencies
- Each context section clearly labelled with source type
- Include explicit instruction: 'If context is insufficient to diagnose — output confidence < 0.4'

- ☐ System prompt written and versioned in code
- ☐ DiagnosisOutput JSON schema defined and validated with Zod
- ☐ Token counting implemented to prevent context overflow

Day 17 Vector Store + Similar Incident Search [DIAGNOSIS]

Morning — pgvector setup

```
pnpm add pgvector
```

- Enable vector similarity search on incidents table

```
CREATE INDEX ON incidents USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 100);
```

- EmbeddingService: wraps OpenAI text-embedding-3-small
- Input: incident title + service + alert labels (compact, not full logs)
- Output: 1536-dimension vector stored in incidents.embedding
- Cost: text-embedding-3-small = \$0.02 per 1M tokens — essentially free

Afternoon — Similarity search

```
// Find top 3 most similar past incidents
SELECT id, title, status, mttr_seconds,
  1 - (embedding <=> $1) AS similarity
FROM incidents
```

```
WHERE tenant_id = $2
  AND status = 'resolved'
  AND id != $3 -- exclude current incident
ORDER BY embedding <=> $1
LIMIT 3;
```

- Similarity threshold: only include if similarity > 0.75
- For each similar incident: include title, root cause, action taken, MTTR
- This gives LLM institutional memory — 'this happened before and X fixed it'

- ☐ Embeddings generated and stored for test incidents
- ☐ Similarity search returning relevant past incidents
- ☐ Similarity search completing in < 50ms

Day 18 LLM Agent — Implementation [DIAGNOSIS]

Morning — OpenAI integration

```
pnpm add openai
```

```
const response = await openai.chat.completions.create({
  model: 'gpt-4o',
  messages: [
    { role: 'system', content: SYSTEM_PROMPT },
    { role: 'user', content: buildContextPrompt(context) }
  ],
  response_format: { type: 'json_object' },
  temperature: 0.1, // low temperature — factual, not creative
  max_tokens: 1000,
});
```

Afternoon — Citation enforcement validator

This is the non-negotiable safety layer. Before any diagnosis reaches the confidence scorer:

- Parse JSON response from LLM
- For each evidence item: verify the reference string actually exists in the context passed to the LLM
- If reference not found in context → mark that evidence item as INVALID
- If > 30% of evidence items are invalid → downgrade confidence to max 0.35 (escalate tier)
- Log all citation failures for monitoring

```
function validateCitations(diagnosis: DiagnosisOutput, context: DiagnosisContext):
ValidationResult {
  const contextText = serializeContext(context);
  const failures = diagnosis.evidence.filter(e =>
    !contextText.includes(e.reference)
  );
  const failureRate = failures.length / diagnosis.evidence.length;
  return { valid: failureRate < 0.3, failureRate, failures };
}
```

- ☐ LLM agent calling OpenAI and returning structured JSON
- ☐ Citation validator checking all evidence references
- ☐ Hallucinated citations downgrading confidence correctly
- ☐ Low temperature set — responses are consistent and factual

Day 19 Confidence Scorer + Diagnosis Storage [DIAGNOSIS]

Morning — Confidence scorer

The LLM outputs a confidence score. We adjust it based on evidence quality before using it to determine action tier.

```
function scoreConfidence(
  llmScore: number,
  citationResult: ValidationResult,
  context: DiagnosisContext
): number {
  let score = llmScore;
  // Penalise citation failures
  score *= (1 - citationResult.failureRate);
  // Boost if deploy correlation found (strong signal)
  if (context.recentDeploy) score = Math.min(score + 0.1, 1.0);
  // Boost if similar incident found with same resolution
  if (context.similarIncidents.length > 0) score = Math.min(score + 0.05, 1.0);
  // Penalise if no logs available
  if (context.logs.length === 0) score *= 0.7;
  return Math.round(score * 1000) / 1000;
}
```

Afternoon — Diagnosis stored + incident updated

- Save diagnosis to diagnoses table with all fields
- Update incident status: 'detecting' → 'diagnosing' → 'acting'
- Embed incident (title + hypothesis) and store in incidents.embedding
- Emit status update via Redis pub/sub for dashboard real-time updates

- ☐ Confidence scorer applying all adjustments correctly
- ☐ Diagnosis saved to DB with citations
- ☐ Incident embedding generated and stored
- ☐ Incident status updated through lifecycle

Day 20 Diagnosis E2E + Prompt Iteration [DIAGNOSIS]

Full day — Quality testing

Run real incident scenarios through the full diagnosis pipeline. Evaluate output quality. Iterate on the system prompt.

Test scenarios to run:

- Scenario 1: High CPU + recent deploy → expect deploy correlation in hypothesis
- Scenario 2: 502 errors + downstream dependency unhealthy → expect dependency root cause
- Scenario 3: Memory leak pattern in logs → expect OOM hypothesis
- Scenario 4: No context available (no logs, no metrics) → expect low confidence + escalation
- Scenario 5: Same incident as a resolved past incident → expect high confidence from similar_incidents

Evaluation criteria per scenario:

- Is the root cause hypothesis correct?
- Are all citations real and present in the context?
- Is the confidence score appropriate to evidence quality?
- Is the recommended action specific and executable?
- Iterate system prompt until 4/5 scenarios produce trustworthy output
- Document all prompt versions with their test results

- ☐ 5 test scenarios run end-to-end
- ☐ 4/5 producing correct, cited hypotheses
- ☐ All citation validation checks passing

Do not move to Phase 3 until diagnosis quality is trustworthy. A wrong auto-execute action on a production system destroys user trust permanently.

PHASE 3: Action Layer — Confidence Gating, Auto-Execute, Rollback Week 5-6 — Days 21-30

Goal: SRE.ai takes appropriate action based on diagnosis confidence. High confidence → auto-execute. Medium → draft for human approval. Low → escalate with full context.

Day 21 Action Router + Slack Integration [ACTION]

Morning — Action router

```
function routeAction(confidence: number, tenantConfig: TenantConfig): ActionTier {
  const thresholds = tenantConfig.confidenceThresholds ?? {
    autoExecute: 0.85,
    draft: 0.60,
  };
  if (confidence >= thresholds.autoExecute) return 'auto_execute';
  if (confidence >= thresholds.draft) return 'draft';
  return 'escalate';
}
```

- Default thresholds: auto-execute > 0.85, draft > 0.60, escalate < 0.60
- Tenant can override per service (payment-service: always escalate)
- Service-level override stored in integrations config

Afternoon — Slack integration

```
pnpm add @slack/web-api
```

- SlackNotifier service with rich Block Kit message formatting
- Incident opened message: title, severity, service, status
- Diagnosis complete message: hypothesis, confidence score, evidence summary, action taken
- Escalation message: full context packet + 'Action Required' CTA
- Resolution message: root cause, action that fixed it, MTTR
- All messages sent to tenant-configured channel

- ☐ Action router correctly tiering by confidence
- ☐ Per-service threshold override working
- ☐ Slack messages sending for all incident lifecycle events
- ☐ Slack Block Kit formatting rendering correctly

Day 22 Auto-Execute Actions — v1 Set [ACTION]

v1 auto-execute actions are deliberately conservative. Only actions that are safe, reversible, and have no permanent side effects.

v1 safe auto-execute actions:

Action	Trigger condition
Restart ECS service	Service health check failing + no deploy in last 30 min
Scale ECS service (add 1 task)	CPU > 85% for > 5 min + at capacity
Flush Redis cache key pattern	Cache poisoning detected in logs
Trigger GitHub Actions re-deploy	Failed deploy detected as root cause
Send Slack alert (always safe)	Any escalation

```
pnpm add @aws-sdk/client-ecs @aws-sdk/client-auto-scaling
```

- Each action implemented as a typed ActionHandler class
- ActionHandler.validate() — pre-checks before executing (is service in correct state?)
- ActionHandler.execute() — performs the action
- ActionHandler.rollback() — undoes the action
- All actions logged to actions table with full payload and outcome

- ☐ ECS restart action working against real AWS ECS
- ☐ ECS scale action working
- ☐ All actions logged to DB
- ☐ Each action has a working rollback method

Day 23 Rollback System + Audit Log [ACTION]

Morning — One-click rollback

```
POST /incidents/:id/actions/:actionId/rollback
```

- Any auto-executed action can be rolled back within 1 hour
- Rollback calls ActionHandler.rollback() with original payload
- Rollback logged as new action record with action_type = 'rollback'
- After rollback: incident status → 'escalated' (needs human attention)
- Slack notification: 'Auto-execute action rolled back — human intervention required'

Afternoon — Audit log

- Every state change in the system emits an AuditEvent
- AuditEvent: tenantId, incidentId, actor (system or userId), action, before, after, timestamp
- Stored in audit_logs table — never deleted, append-only
- Exportable as JSON or CSV per incident for post-mortem

- ☐ Rollback working for all v1 auto-execute actions
- ☐ Rollback triggering escalation notification
- ☐ Audit log capturing all state changes

Day 24 PagerDuty Escalation + Draft PR [ACTION]

Morning — PagerDuty integration

```
pnpm add node-pagerduty
```

- Create PagerDuty incident via Events API v2
- Incident body: SRE.ai hypothesis, confidence, evidence summary, recommended action
- Severity mapping: P1 → critical, P2 → error, P3 → warning
- Include direct link to SRE.ai incident dashboard in PagerDuty body
- Resolve PagerDuty incident when SRE.ai marks incident resolved

Afternoon — Draft PR via GitHub

- For medium-confidence incidents where diagnosis suggests a code fix
- Create GitHub branch: sreai/fix-{incident-id}
- Open draft PR with: hypothesis, evidence, suggested fix description
- Assign PR to service owner (from integration config)
- PR is a draft — engineer reviews and merges or closes

- ☐ PagerDuty incident created on escalation
- ☐ PagerDuty incident resolved when SRE.ai resolves
- ☐ Draft PR created with full context

Day 25-26 Human Approval Flow + Action E2E [ACTION]

Day 25 — Human approval for draft actions

- Draft actions sent to Slack with Approve / Reject buttons (Slack interactive components)
- Approval triggers the action execution
- Rejection logs reason (optional) and escalates to PagerDuty
- Approval expires after 30 minutes — auto-escalates if no response
- Slack webhook endpoint to receive button interactions

Day 26 — Full action layer E2E test

- Test: high confidence diagnosis → auto-execute → verify action taken → verify logged
- Test: medium confidence → Slack draft message → approve → execute
- Test: low confidence → PagerDuty incident created with full context
- Test: rollback → action undone → incident re-escalated
- Test: service-level override (payment service always escalates regardless of confidence)

- ☐ All 3 action tiers working end-to-end
- ☐ Human approval flow working via Slack
- ☐ Rollback tested and verified
- ☐ Audit log complete for each test scenario

Phase 3 complete. SRE.ai now detects, diagnoses, and acts. This is the full core loop. Everything from here is making it better and more observable.

PHASE 4: Memory & Learning — Incident Store, Post-Mortems, Runbooks Week 7 — Days 27-32

Day 27-28 Auto-Generated Post-Mortems [MEMORY]

Post-mortem generator

Every resolved incident automatically generates a post-mortem in markdown. No engineer needs to write it from scratch.

```
# Post-Mortem: {incident.title}
**Date:** {incident.openedAt}
**Severity:** {incident.severity}
**Service:** {incident.serviceName}
**MTTR:** {incident.mttrSeconds}s

## Timeline
{auto-generated from audit_log events}

## Root Cause
{diagnosis.hypothesis}

## Evidence
{diagnosis.evidence as bullet list with citations}

## Action Taken
{action.actionTaken} – Outcome: {action.outcome}

## Prevention
{LLM-generated prevention suggestions based on root cause}
```

- Post-mortem generated on incident resolution
 - Stored as markdown in S3 (free tier: 5GB)
 - Linked from incident record in DB
 - Downloadable from dashboard
 - Prevention section: one more LLM call with prompt: 'Given this root cause, suggest 3 specific prevention measures'
-
- ☐ Post-mortem generated on every resolved incident
 - ☐ Stored in S3 and linked from incident
 - ☐ Prevention suggestions included

Day 29-30 Resolution Pattern Learning + Runbook Generator [MEMORY]

Resolution pattern extraction

- After each resolution: extract pattern → {alert_signature, root_cause_type, action_taken, outcome, confidence}
- Store in resolution_patterns table
- Pattern matching: when new incident arrives, check if signature matches a known pattern
- Known pattern with high historical success rate → boosts confidence score

Runbook auto-generator

- Trigger: same incident type resolved the same way 3+ times
- Generate runbook: title, trigger conditions, diagnosis steps, resolution steps, prevention
- Runbook stored as markdown, linked to service and alert type
- Runbook shown to engineer on escalation: 'We've seen this before. Here's the runbook.'

- ☐ Resolution patterns extracted on every resolution
- ☐ Pattern matching boosting confidence on known incidents
- ☐ Runbook generated after 3 similar resolutions

Daily email digest (via AWS SES)

- Sent every day at 9am tenant timezone
- Contents: incidents yesterday, resolutions, auto-executions, MTTR trend, top recurring issue
- Scheduled via BullMQ cron job
- HTML email template — clean, readable on mobile

Analytics queries (for dashboard Phase 5)

- MTTR by service (last 30 days)
- Incidents by severity over time
- Auto-resolve rate (% resolved without human)
- Top 5 recurring incident types
- Confidence score distribution
- All queries pre-written as TypeORM query builder — ready for dashboard

- ☐ Daily digest sending at correct time
- ☐ All 6 analytics queries tested and returning correct data

PHASE 5: Dashboard + Polish + AWS Deploy *Week 8-10 — Days 33-50*

Day 33-36 Next.js Dashboard [DASHBOARD]

```
npx create-next-app@latest dashboard --typescript --tailwind --app
```

Pages to build:

- / — Dashboard home: active incidents, recent resolutions, MTTR trend chart
- /incidents — List with filters: status, severity, service, date range
- /incidents/[id] — Incident detail: timeline, diagnosis, citations, actions, audit log
- /integrations — Connect monitoring tools, manage API keys
- /runbooks — Auto-generated runbooks list
- /analytics — MTTR charts, auto-resolve rate, recurring issues
- /settings — Confidence thresholds, notification preferences, team members

Key UI components:

- IncidentCard — severity badge, status, service, MTTR, quick actions
- DiagnosisPanel — hypothesis, confidence meter, evidence list with citations
- TimelineView — incident lifecycle events from audit log
- ConfidenceMeter — visual bar showing confidence + threshold markers
- CitationList — each evidence item with source type and reference

Real-time updates via WebSocket:

- NestJS Gateway (Socket.IO) emitting incident status changes
- Dashboard subscribes per-incident — live status without refresh

- Redis pub/sub as transport for multi-instance support

- ☐ All pages built and rendering real data
- ☐ Incident detail showing full diagnosis with citations
- ☐ Real-time status updates working
- ☐ Mobile-responsive layout

Day 37-40 Onboarding Flow + Integration Wizard [DASHBOARD]

Onboarding — first 5 minutes experience

- Step 1: Create account (email + password or GitHub OAuth)
- Step 2: Name your first service (e.g. 'auth-service')
- Step 3: Choose your alert source — Prometheus, Sentry, Grafana, CloudWatch
- Step 4: Copy webhook URL + API key
- Step 5: Send a test alert — see it flow through to diagnosis in real time
- Step 6: Connect Slack for notifications

The onboarding goal: user sees a real diagnosis within 10 minutes of signup. That first 'aha moment' is everything for retention.

Integration wizard:

- Step-by-step instructions per integration type (Prometheus, Sentry, etc.)
 - Copy-paste ready config snippets (Prometheus alertmanager.yml, Sentry webhook settings)
 - Test button: sends synthetic alert, shows result in real time
 - Status indicator: green/red per integration
- ☐ Onboarding flow completing in < 10 minutes
 - ☐ Test alert showing real-time result in dashboard
 - ☐ Integration wizard with copy-paste configs for each source

Day 41-44 AWS Production Deploy [DEPLOY]

AWS Free Tier architecture:

Component	AWS Service
API Gateway + Services	EC2 t2.micro (Docker + docker-compose)
PostgreSQL	RDS PostgreSQL db.t3.micro
Redis	ElastiCache cache.t3.micro
SQS Queues	SQS (free tier: 1M req/month)
File Storage	S3 (post-mortems, logs)
Logs	CloudWatch Logs
Domain + SSL	Route 53 + ACM (SSL free)
Email	SES (62k/month free from EC2)

Deploy steps:

- VPC with public subnet (API) + private subnet (RDS, Redis)
- Security groups: EC2 allows 80/443, RDS allows 5432 from EC2 only
- RDS: enable automated backups (free up to 20GB)
- EC2: install Docker, copy docker-compose.prod.yml, run services
- Route 53: point domain to EC2 Elastic IP
- ACM: issue SSL cert for domain, configure Nginx reverse proxy
- GitHub Actions: SSH deploy on push to main branch

Free tier EC2 t2.micro has 1GB RAM. Running all 4 NestJS services + Nginx will be tight. Consolidate into 2 processes if needed: ingestion + diagnosis in one, action + api-gateway in another.

- ☐ All services running on EC2
- ☐ RDS and ElastiCache in private subnet
- ☐ Domain live with HTTPS
- ☐ GitHub Actions deploying on push to main
- ☐ Health check endpoint returning 200

Day 45-48 Testing, Load Testing, Security Hardening [POLISH]

Day 45 — Full E2E test suite

- E2E test: full incident lifecycle — alert → dedup → enqueue → diagnose → act → resolve → post-mortem
- Test all webhook sources: Prometheus, Sentry, Grafana, CloudWatch
- Test all action tiers: auto-execute, draft, escalate
- Test rollback for each auto-execute action
- Test multi-tenant isolation: tenant A cannot access tenant B data

Day 46 — Load testing

- Target: 50 concurrent webhook requests/second sustained for 60 seconds
- Target: diagnosis job processing < 30 seconds end-to-end
- Target: dashboard loads in < 2 seconds on EC2 t2.micro
- Fix any bottlenecks found

Day 47-48 — Security hardening

- OWASP Top 10 review: SQL injection, XSS, CSRF, broken auth, sensitive data exposure
 - Rate limiting: 100 req/min per IP on API, 1000 req/min on webhook endpoints
 - Helmet.js: security headers on all responses
 - Input sanitisation: all webhook payloads sanitised before LLM (prompt injection prevention)
 - API key rotation endpoint tested
 - Penetration test: try to access another tenant's data with a valid JWT
-
- ☐ Full E2E suite passing
 - ☐ Load test meeting targets
 - ☐ Rate limiting working
 - ☐ Multi-tenant isolation verified under adversarial test

Day 49-50 Beta Prep + First User [LAUNCH]

Day 49 — Onboarding docs + landing page

- README: clear setup guide, supported integrations, FAQ
- Landing page: hero (value prop), how it works (3 steps), pricing, sign up CTA
- Docs site: integration guides for each source (Prometheus, Sentry, Grafana, CloudWatch)
- Status page: uptime monitoring via UptimeRobot (free)

Day 50 — First real user

- Identify 1 startup founder/CTO from your network
- Offer free access in exchange for 30-minute feedback call
- Walk through onboarding together on a call
- Watch where they get confused — that's your UX debt list
- Collect: 'what would make you pay for this?' answer

- ☐ Landing page live at your domain
- ☐ Docs site with all integration guides
- ☐ First user onboarded and processing real alerts
- ☐ Feedback collected and prioritised

Post-MVP: Week 11+ Priorities

After MVP and first users, work in this order:

Priority	Feature	Why now
P0	Fix bugs from first user feedback	Retention > acquisition always
P0	Kubernetes support (pod restart, rollout)	Most startups are on k8s
P1	Stripe billing integration	Can't grow without payments
P1	Datadog alert source	Popular monitoring tool
P1	Anomaly detection (pre-alert)	Moves product from reactive to proactive
P2	Linear/Jira ticket creation	Reduces manual work on escalations
P2	Multi-service dependency map	Visual infra understanding
P3	Fine-tuned smaller LLM	Reduce OpenAI cost at scale

Recommended Daily Routine (Full-Time)

Time	Block
9:00 - 9:30	Review yesterday: what's done, what's blocked, update checklist
9:30 - 1:00	Deep work block 1 — primary build task for the day

1:00 - 2:00	Lunch + break — step away from screen
2:00 - 5:30	Deep work block 2 — continue or secondary task
5:30 - 6:30	Testing + debugging what you built today
6:30 - 7:00	Commit, push, update checklist, plan tomorrow

One rule: never end a day without a working commit. Even if it's a rough implementation — commit it. Forward momentum is everything on a solo project.

Running Cost Tracker

Service	Monthly cost
AWS Free Tier (EC2 + RDS + Redis + SQS)	\$0 (first 12 months)
OpenAI API (GPT-4o, 100 diagnoses/month)	~\$5-15/month
OpenAI API (embeddings)	~\$0.10/month (essentially free)
Domain (.ai domain)	~\$40/year (~\$3.5/month)
Sentry free tier	\$0
GitHub free tier	\$0
UptimeRobot free tier	\$0
Total until first customer	~\$20-25/month

Total cost to MVP: under \$200. When free tier expires at 12 months, AWS costs jump to ~\$80-100/month. By then you should have paying customers covering this.

50 days. Solo. Full-time.

From idea to real product with real users.

Start Day 1 today.